Concluding Topic

- Diagnosing and Fixing Software Projects

Topic # 15

- Reference Text

Lack of Leadership

- It takes more than a talented and motivated team to make a successful project.
- Lack of leadership manifests itself in the team members suffering from:
  - Tunnel vision
  - Over-reliance on gut instincts
  - Repeated false starts in the project

The Mid-Course Correction

- A change in project priorities throws the team into disarray
- This usually comes from a lack of understanding of the scope of the project
- When the engineers don't understand the users' and stakeholders' needs, they build the wrong software

The Detached Engineering Team

- There is an artificial wall between the people who build the software and those who need it.

Fixing Planning Problems

- Lack of Leadership, the Mid-Course Correction and the Detached Engineering Team are project planning problems
  - Use a vision and scope document
  - Use a project plan to keep everyone informed about how those needs will be met
  - Use risk planning to keep the plan realistic

Padded Estimates Generate Distrust

- Programmers add extra time to their estimates
  - They may do this because of unknowns
- Project managers and senior managers quickly figure this out, and start to question individual estimates

Self-Fulfilling Prophecy

- A project manager under pressure simply imposes a deadline, and creates unrealistic estimates that meet it
- The team works nights and weekends to meet the deadline

Fixing Estimation Problems

- Padded estimates and the self-fulfilling prophecy are estimation problems
  - Adopting a repeatable estimation process like Wideband Delphi can help fix them
  - By writing down assumptions, the team can handle risks without padding their time – and even avoid the risks altogether

Working Backwards From a Deadline

- Project managers approach a non-negotiable deadline for a project by working backwards
  - They shorten the tasks in the schedule or cutting them entirely until everything fits
  - When the schedule gets tight, any non-programming activities are cut and the software is released before it's finished

Misunderstood Predecessors

- The project manger does not take the time to understand how tasks depend on each other
- Problems are discovered partway through the project one task can't be started because it depends on another
- Delays cascade through the project, getting increasingly worse

Fixing Scheduling Problems

- Working backwards from a deadline and misunderstood predecessors are symptoms of underlying scheduling problems
  - They can be avoided by adopting good planning and estimation practices and creating a project schedule
  - Schedule techniques like critical path analysis can help spot problems early on

Problems Are Found Too Late

- There are preventable defects in the software that aren't caught until late in the project
  - The team may misunderstand a need
  - Requirements may be missed or incorrect
  - The design may be difficult to use or fail to take all of the features into account

Big, Useless Meetings

- A project manager who has previously been burned by problems that were found too late is determined to avoid falling into the same trap
  - He calls a big meeting with everyone who could possibly have input
  - The meeting drags on for hours, without making any real progress
  - Eventually, everyone gives up and goes back to the way they did things before

The Indispensable "Hero"

- One "critical" person is seen as the clear top programmer, and all important work is sent through him
  - He may have a unique skill or experience
  - He is always working long hours – and causing bottlenecks

Fixing Review Problems

- Problems that are found too late, big useless meetings, and the indispensable "hero" are problems which can be solved with reviews
  - Reviews can catch defects early, when they are cheaper to fix
  - A review meeting only includes the people necessary for the work to be done
  - Reviews – especially code reviews – can help the "hero" spread his expertise and knowledge

Iteration Abuse

- Iteration can be a useful tool, but it is often abused
- The team uses iteration as a "guessing game"
  - Programmers deliver build after build; users and stakeholders make small changes to each build
  - Programmers like it because they can dive in
  - Users and stakeholders like it because they don't have to read documents or think about their needs

Scope Creep

- After the programming has started, users and stakeholders make changes
- Each change is easy to describe, so it sounds "small" and the programmers agree to it
- Eventually, the project slows to a crawl

Fixing Requirements Problems

- When software requirements are not gathered and specified before the software is developed, it causes scope creep and the team resorts to iteration abuse.
  - The team can adopt software requirements engineering practices to write down most of the changes before the work begins
  - A change control process gives them a handle on the few changes that remain

Haunted by Ghosts of Old Problems

- Programmers find that old bugs suddenly reappear without warning
  - As the code base grows, it becomes harder to keep control of the source code
  - They may use a shared folder to store source code, but occasionally old copies of files are copied over newer ones

Broken Builds

- The programmers deliver a build which does not work – and the testers can't even begin to test it
  - The programmers get frustrated because they feel that they put a lot of effort into testing the build
  - The testers spend hours or days setting up a test environment, only to redo it for the next build

Spaghetti Code

- Maintaining old code is the least desirable programming job in many organizations
  - Old, highly modified code turns into a twisted mess of execution paths and patches

Fixing Programming Problems

- When the team adopts good programming habits, they can avoid ghosts of old problems, broken builds and spaghetti code.
  - Get control of the source code with version control software like Subversion
  - Use unit tests and test-driven development to increase the quality of the build
  - Use refactoring to keep the code readable

Requirements Haven't Been Implemented

- The team delivers software missing behavior or even entire features

Obvious Bugs Slipped Through

- Inexperienced testers are expected to just "bang on the software"
- Even when experienced testers are used, they are not given time to plan
- Decisions about release readiness are made based on the schedule, rather than the quality

"But It Worked For Us!"

- When a product is not tested in all environments in which it will be used, the tests will be thrown off
- Defects are missed when testers can't adequately replicate the environment in which the software will be used
- Test data may not resemble actual production data

Fixing Testing Problems

- When code is delivered with too few requirements implemented and too many bugs included, the team needs better testing practices.
  - Software testers must be involved in every stage of development
  - Test planning must be given adequate time on the schedule
  - Sufficient budget must be provided for a testing environment.

Common Problems Can Be Avoided!

- Almost everyone has experienced at least a few of these problems.
- We know what causes them, and we have tools, techniques and practices that can fix them.
- All it takes is good project management and sound software engineering… and any project team can do it!

END OF TOPIC 15

-COMING UP!!!!!!
-Revision & Cases
-Semester Exam
-Project Submission & Demo

28